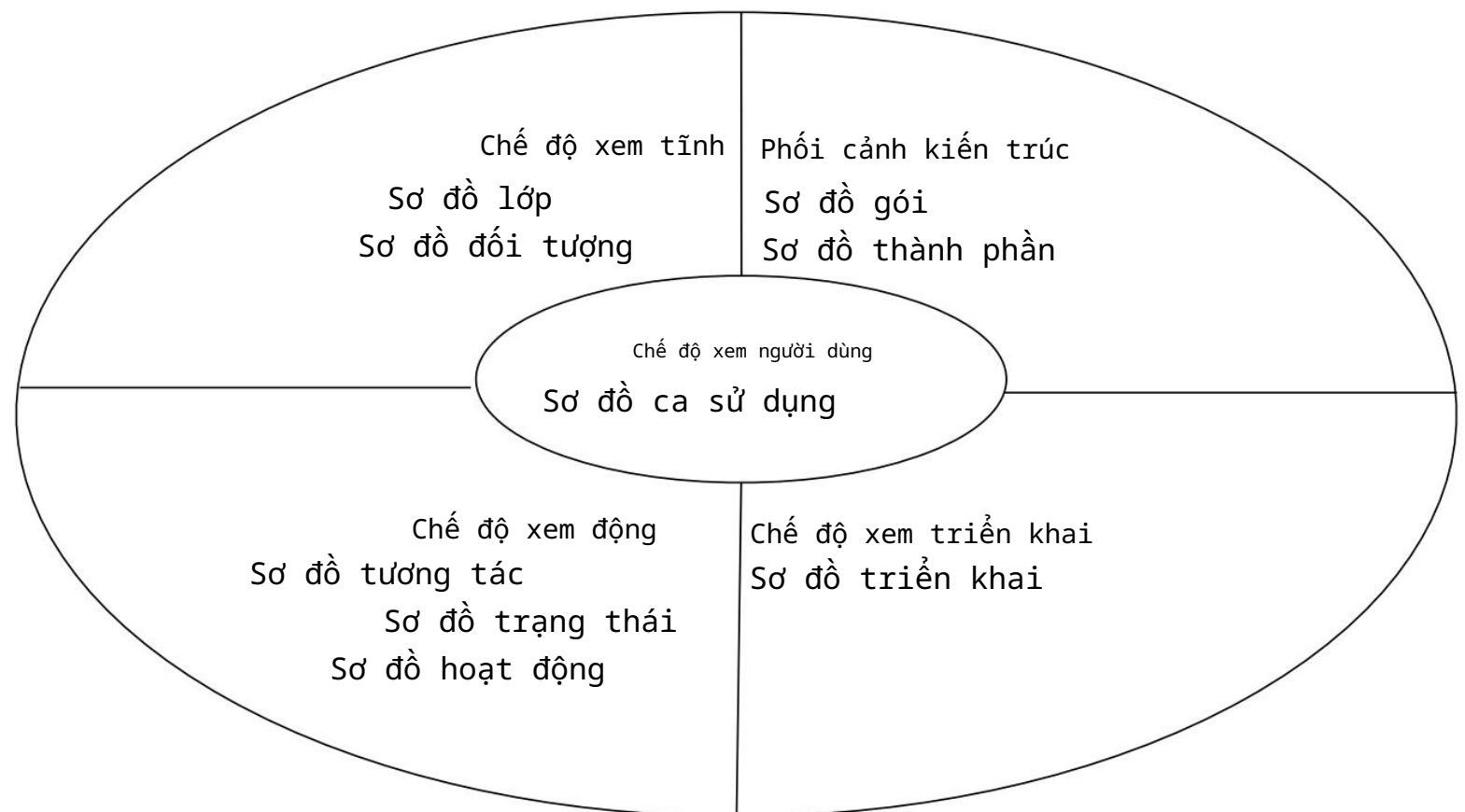


Mô hình kiến trúc & triển khai

Sơ đồ gói Sơ đồ
thành phần Sơ đồ triển khai



Sơ đồ gói

Một gói cho phép nhóm các phần tử liên quan với nhau ■ Một số

lớp liên quan được nhóm lại với nhau thành một gói ■ Một số gói liên quan được nhóm thành

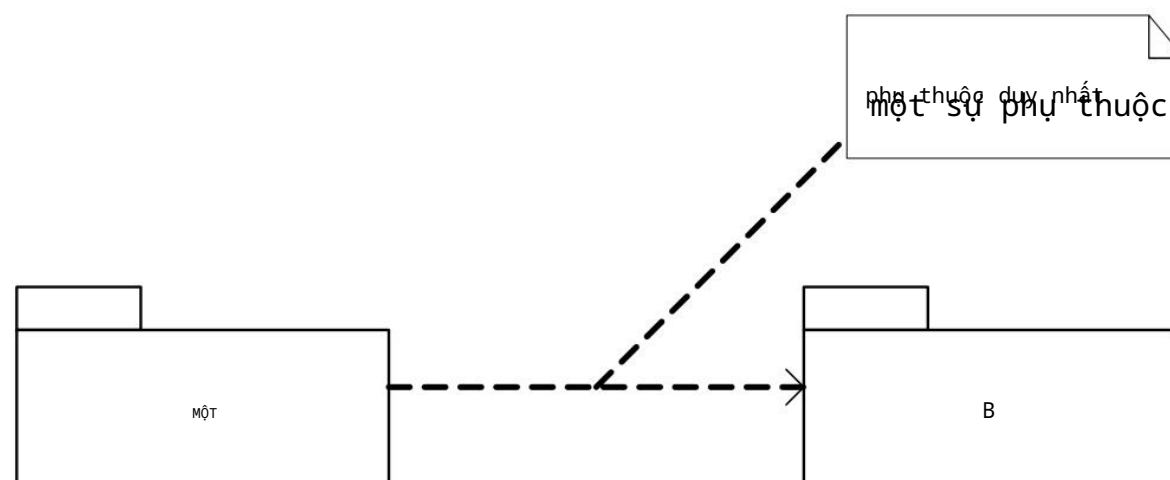
một gói khác Ký hiệu



Sự phụ thuộc ■ Một

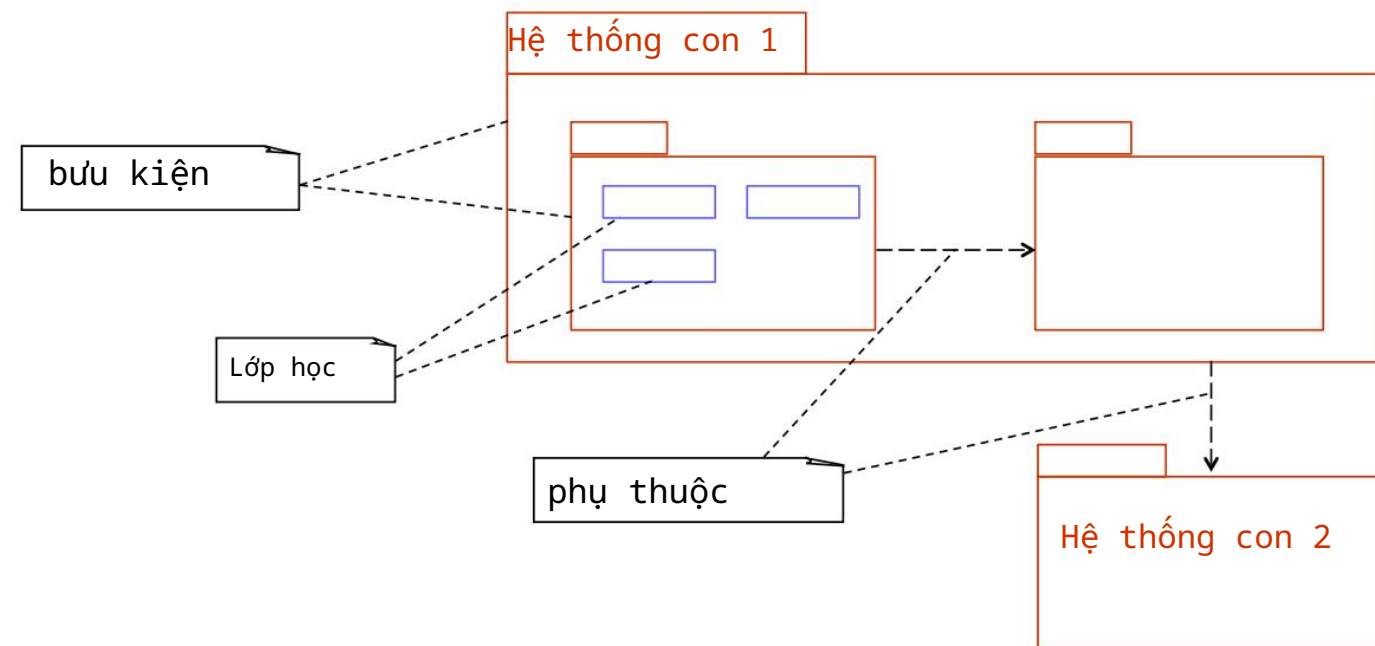
gói có thể phụ thuộc vào một gói khác

Ví dụ: một gói đề cập đến một phần tử của gói khác ■ Ký hiệu



Sơ đồ gói

Ví dụ



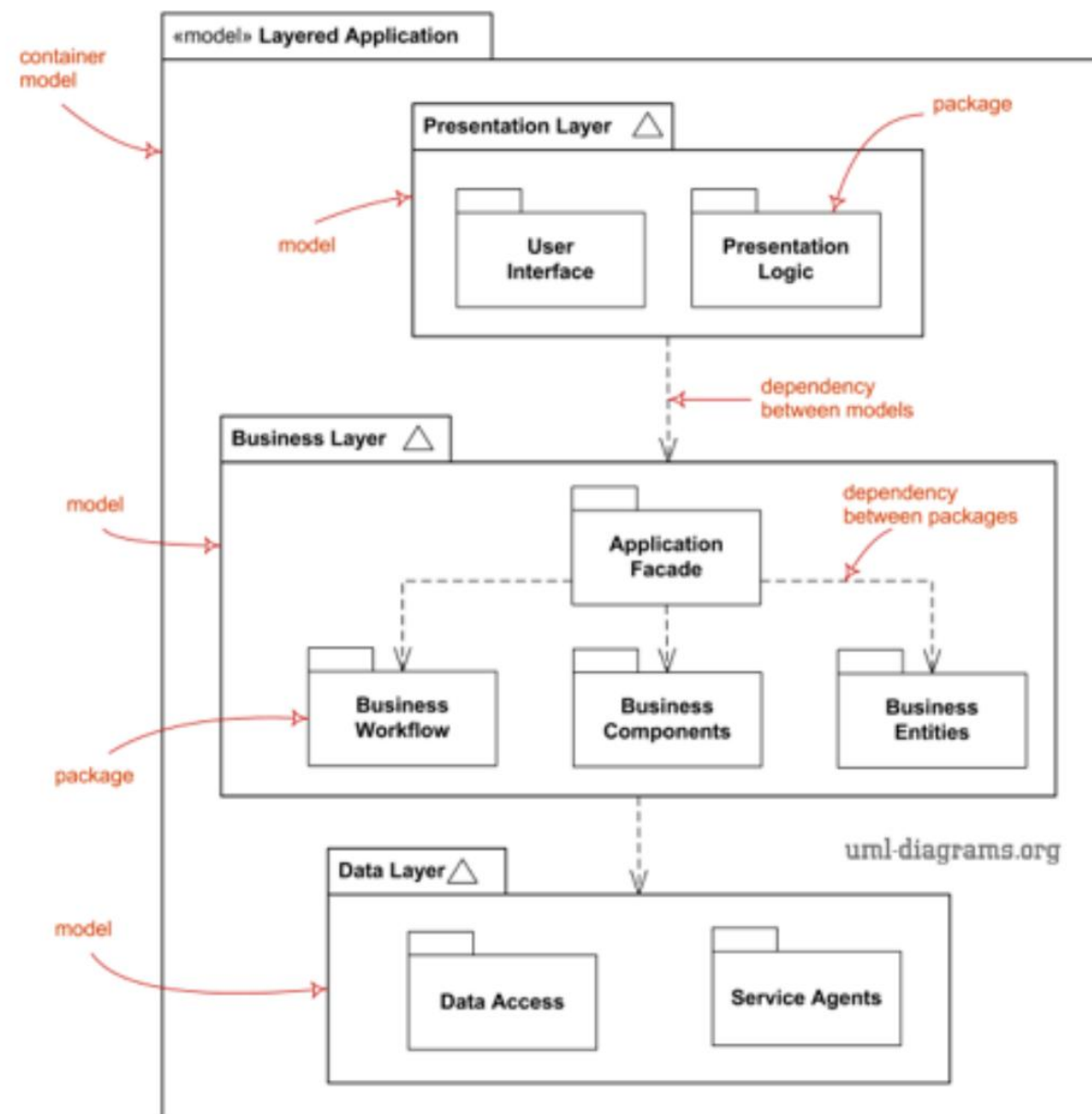
Sơ đồ gói hàng

Tại sao lại là gói

hàng? ■ Dễ quản lý, hiểu và thao tác ■ Giảm độ phức tạp

- Phát triển lặp lại: các nhà phát triển, nhóm khác nhau làm việc đồng thời trên các gói khác nhau

Ví dụ



Sơ đồ gói

Nguyên tắc tổ chức gói ■ Gắn kết chức năng

Các lớp/giao diện được nhóm lại có liên quan chặt chẽ về mục đích, dịch vụ, công tác, chức năng ■ Ví dụ: tất cả các thành phần của gói “thanh toán” đều liên quan đến việc thanh toán sản phẩm



Sơ đồ gói

Nguyên tắc sắp xếp gói hàng

- Sự gắn kết chức năng

Sự gắn kết của một gói được định lượng bằng

$$C = \frac{\text{Số lượng mối quan hệ nội bộ}}{\text{Số phần tử}}$$

- Giá trị C nhỏ có thể nói rằng Gói chứa quá nhiều mục không liên quan, không tốt được tổ chức

Gói hàng không chứa các mục không liên quan với mục đích nhà thiết

kế chẳng hạn như gói “công cụ”

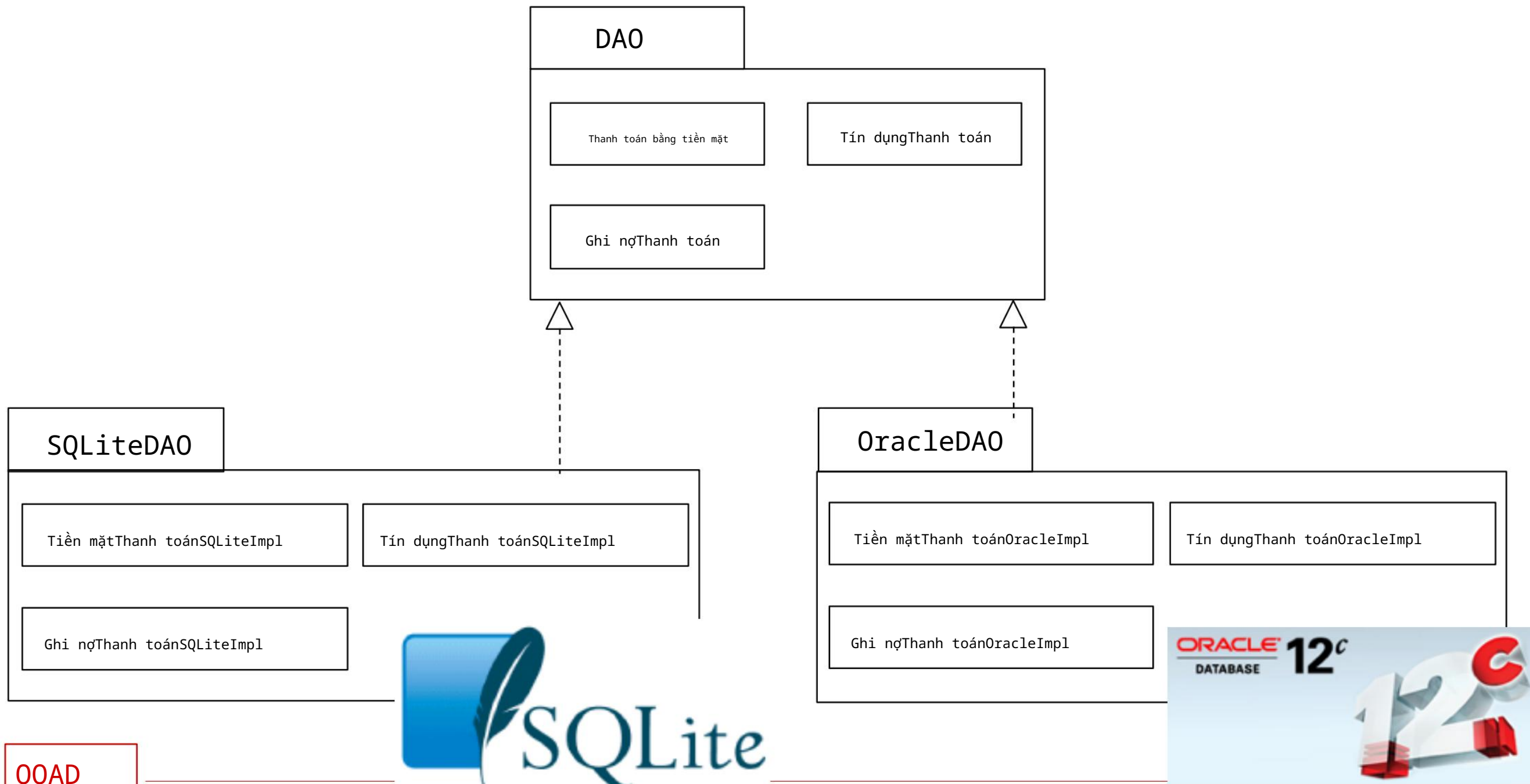
Gói này chứa một tập hợp con các phần tử có tính gắn kết cao, nhưng tổng thể không gắn kết

Sơ đồ gói

Nguyên tắc sắp xếp gói hàng (tiếp theo)

■ Gói giao diện Các giao

diện liên quan được đặt trong một gói Các lớp
triển khai được tách biệt



Sơ đồ gói

Nguyên tắc tổ chức các gói (tiếp theo) ■ Gói các phần tử không ổn

định Các phần tử ổn định được nhóm lại trong một

gói Các phần tử không ổn định được nhóm lại với nhau trong một

gói

■ Đây là những hạng mục thường xuyên được sửa đổi và phân phối lại

■ Để giảm tác động lên các bộ phận ổn định ■ Ví dụ: một gói hàng bao

gồm 20 loại trong đó có 10 loại

thường được sửa đổi và phân phối lại. Tốt nhất nên tách chúng thành hai gói: một gói bao gồm mười lớp ổn định, gói kia không ổn định.

Sơ đồ gói

Nguyên tắc tổ chức các gói (tiếp theo) ■ Càng phụ thuộc thì càng ổn định Gói càng phụ thuộc

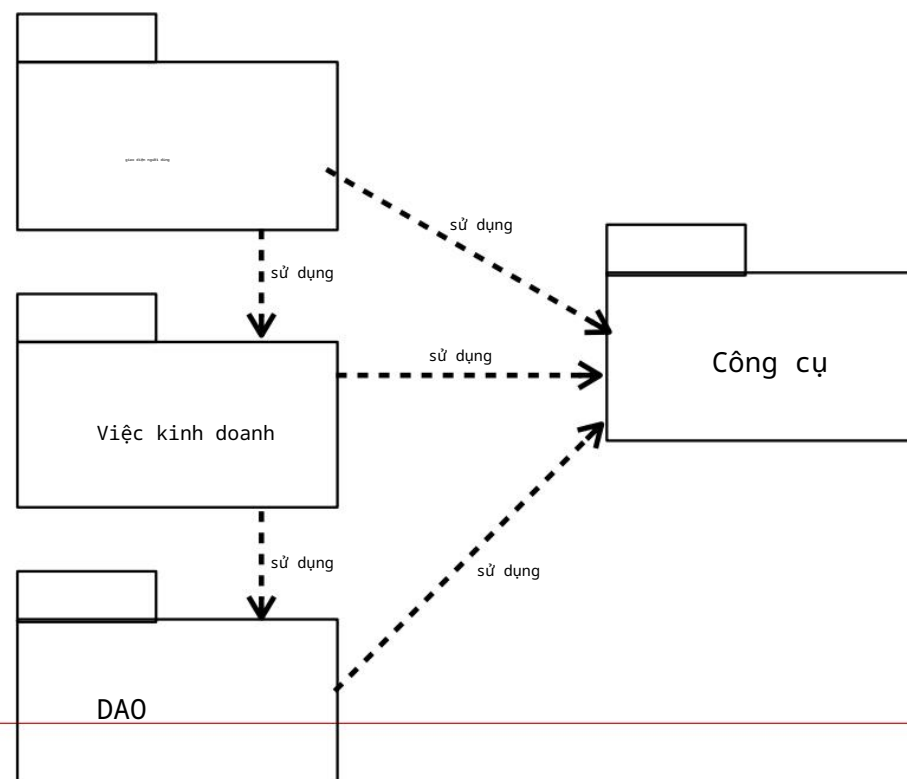
thì càng ổn định ■ Vì những thay đổi trên các gói này có tác động lớn đến các gói khác

gói Ví dụ:

- gói “công cụ” cần phải ổn định.

Một số cách cải thiện tính ổn định của gói hàng

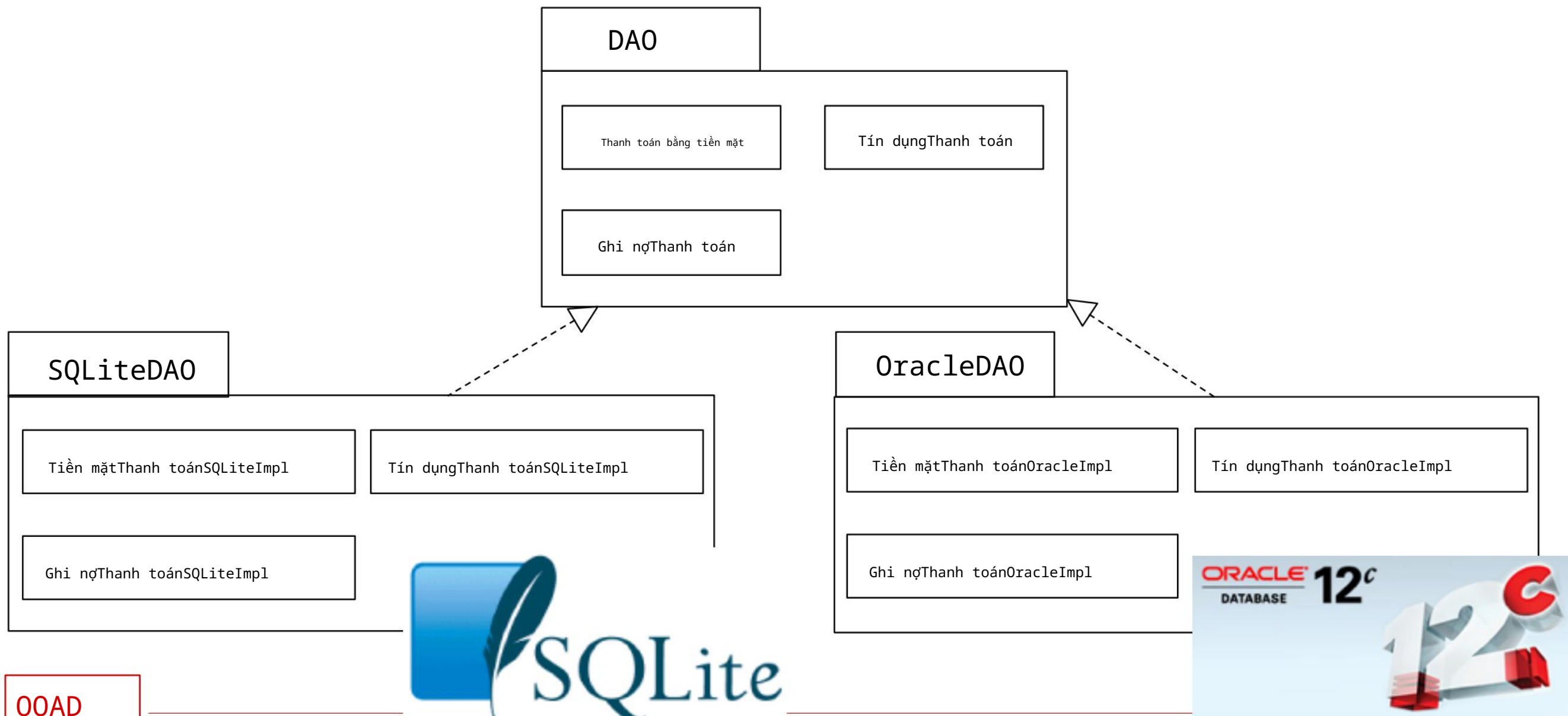
- Nó chỉ chứa các giao diện hoặc các lớp trừu tượng
- Nó không phụ thuộc vào các gói khác, hoặc chỉ phụ thuộc vào các gói rất ổn định
- Nó chứa mã ổn định (được triển khai tốt)



Sơ đồ gói

Nguyên tắc tổ chức các gói (tiếp theo) ■ Gói các phần tử độc lập
 Các phần tử được sử dụng độc lập
 hoặc trong các bối cảnh khác nhau
 được tách thành các gói khác nhau

Ví dụ: SQLite & Oracle dành cho môi trường phát triển/thử nghiệm và sản xuất tương ứng



Sơ đồ thành phần

Sơ đồ thành phần mô hình hóa khung nhìn kiến trúc của hệ thống, triển khai vật lý của hệ thống

- Các lớp mô tả tổ chức logic, trong khi Thành phần mô tả việc triển khai vật lý

Sơ đồ thành phần xác định các mô-đun phần mềm vật lý và mối quan hệ của chúng với nhau

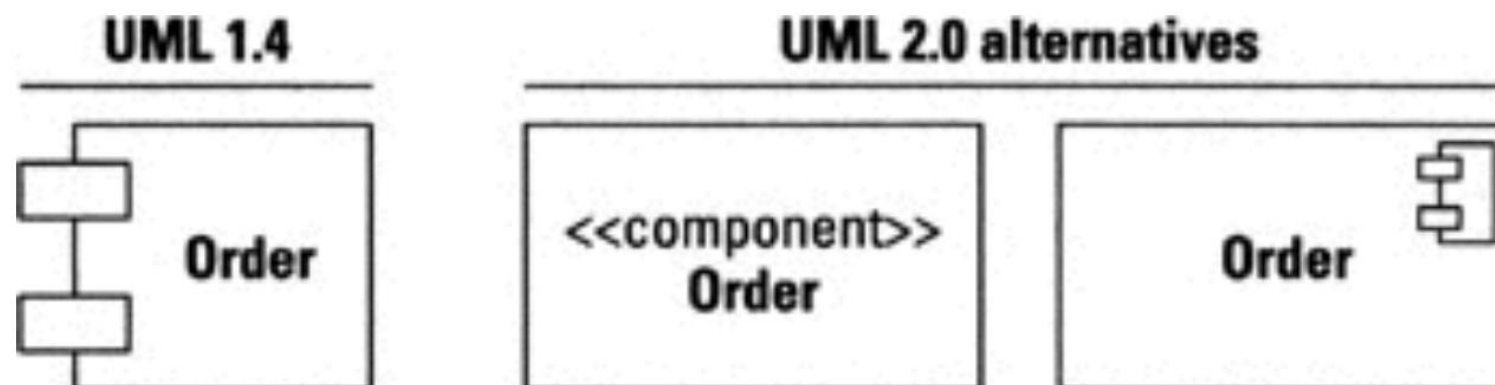
Các thành phần có thể đại diện cho mọi thứ từ một lớp đơn lẻ đến các ứng dụng, hệ thống con và hệ thống

Các tạo phẩm triển khai các thành phần

Các tạo phẩm có thể là bất kỳ loại mã nào có thể nằm trong bất kỳ loại mã nguồn bộ nhớ, tệp nhị phân, tập lệnh, tệp thực thi, cơ sở dữ liệu hoặc ứng dụng

Sự phụ thuộc đại diện cho các loại mối quan hệ tồn tại giữa các thành phần trên sơ đồ thành phần

Ký hiệu



```

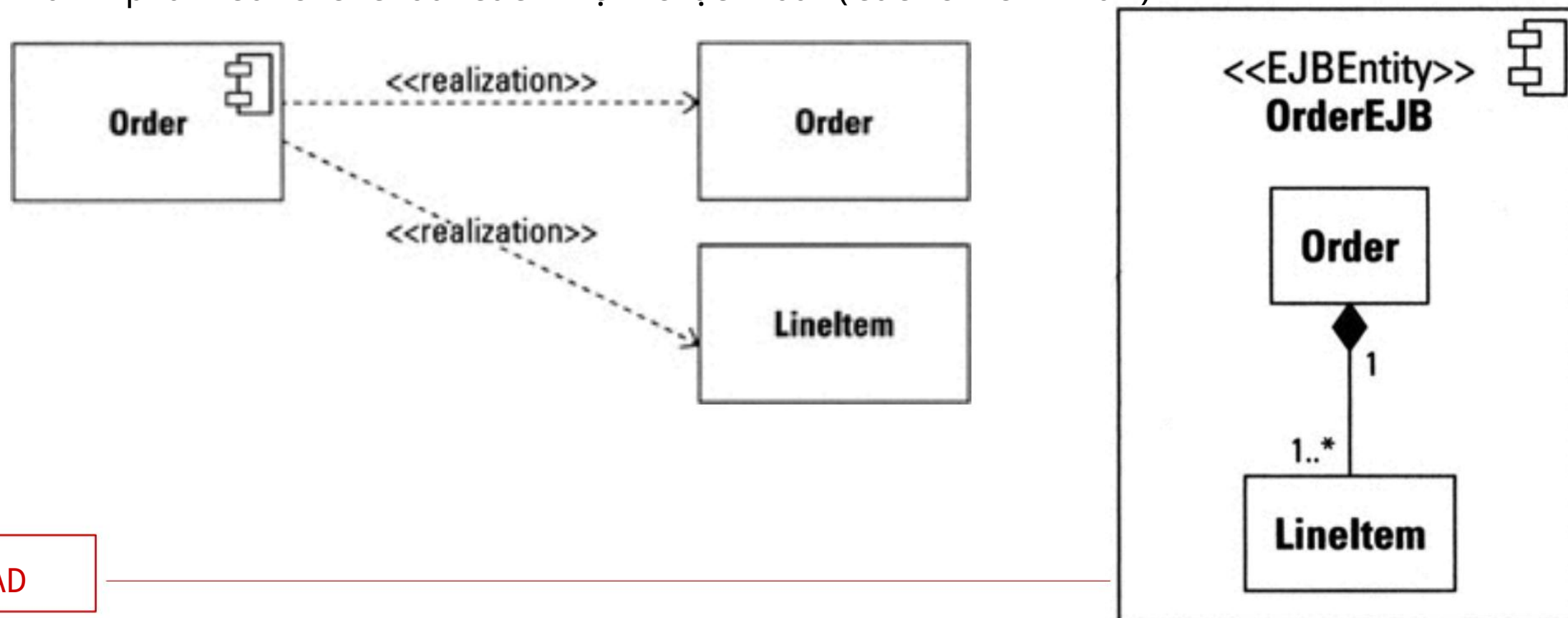
    graph LR
      OrderEJB["<<EJBEntity>>  
OrderEJB"]
      PerformanceEJB["<<EJBEntity>>  
PerformanceEJB"]
      EventEJB["<<EJBEntity>>  
EventEJB"]
      Discount["<<EJBEntity>>  
Discount"]
      VenueEJB["<<EJBEntity>>  
VenueEJB"]

      OrderEJB -.-> PerformanceEJB
      OrderEJB -.-> Discount
      PerformanceEJB -.-> EventEJB
      PerformanceEJB -.-> VenueEJB
  
```

The diagram illustrates the following dependencies:

- OrderEJB** depends on **PerformanceEJB** and **Discount**.
- PerformanceEJB** depends on **EventEJB** and **VenueEJB**.

Thành phần có thể chứa các hiện thực hóa (các triển khai)



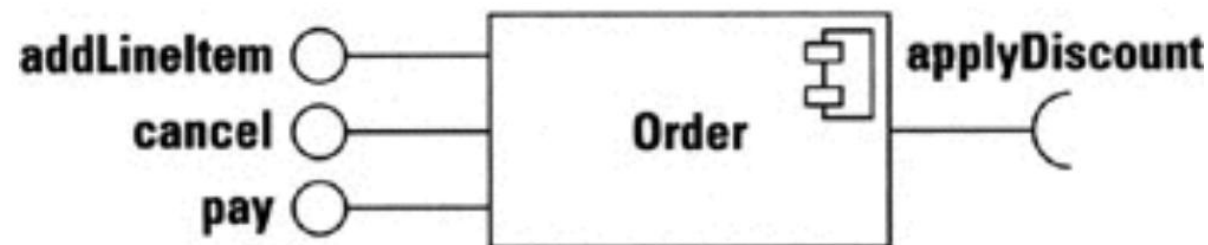
Sơ đồ thành phần

Một tính năng cơ bản của các thành phần là khả năng xác định giao diện

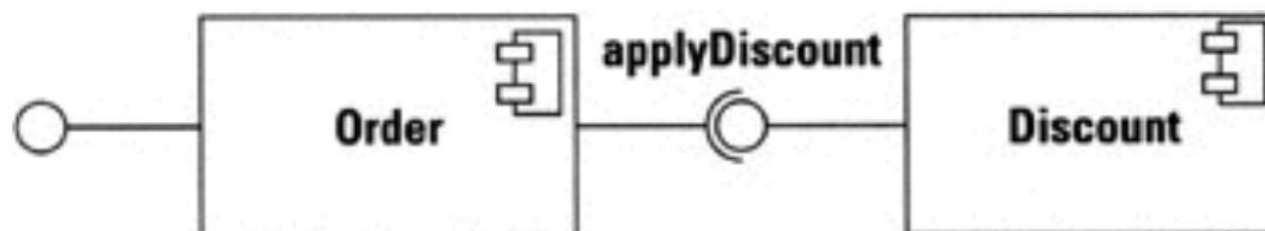
Giao diện có hai loại: bắt buộc và cung cấp

- Giao diện được cung cấp: xác định cách thành phần khác phải yêu cầu quyền truy cập vào dịch vụ được cung cấp
- Giao diện bắt buộc: xác định giao diện chính xác những gì nó cần

Ví dụ: Thành phần đơn hàng cung cấp dịch vụ "thêm chi tiết đơn hàng vào đặt hàng", "hủy đơn hàng" và "thanh toán đơn hàng"; và yêu cầu giảm giá (từ thành phần khác)



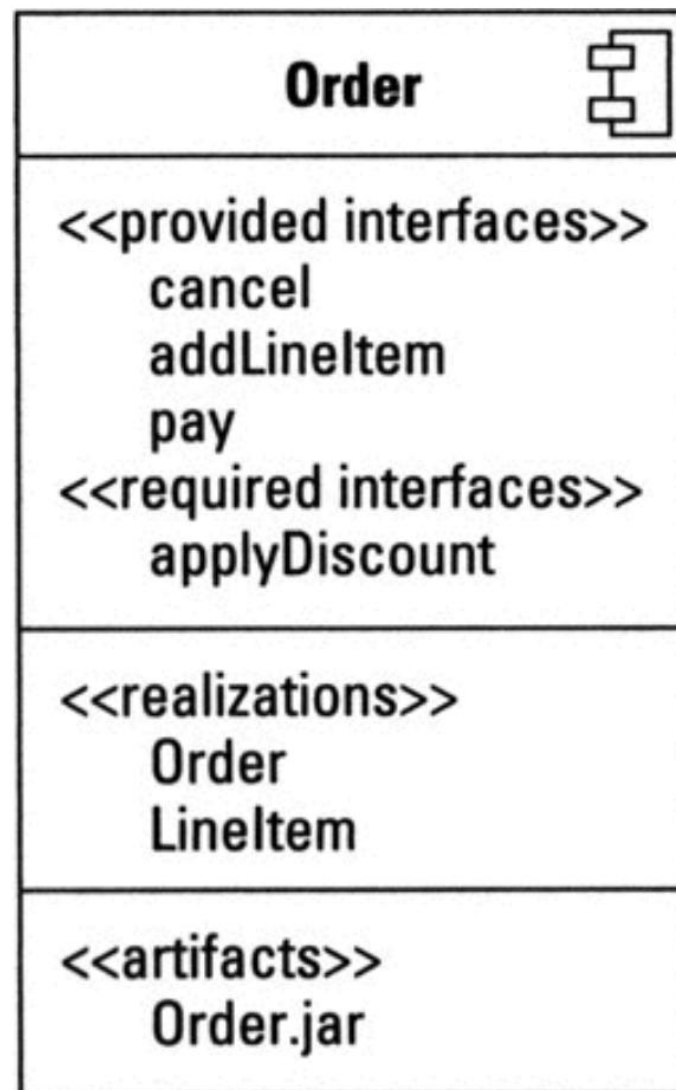
Kết nối các giao diện được yêu cầu và cung cấp để hình thành quan hệ đối tác giữa các thành phần



Sơ đồ thành phần

UML 2.0 cung cấp một cách để thể hiện tất cả thông tin được xác định cho đến nay cho một thành phần, bao gồm các giao diện, cách thực hiện và các tạo phẩm

- chế độ xem hộp màu trắng

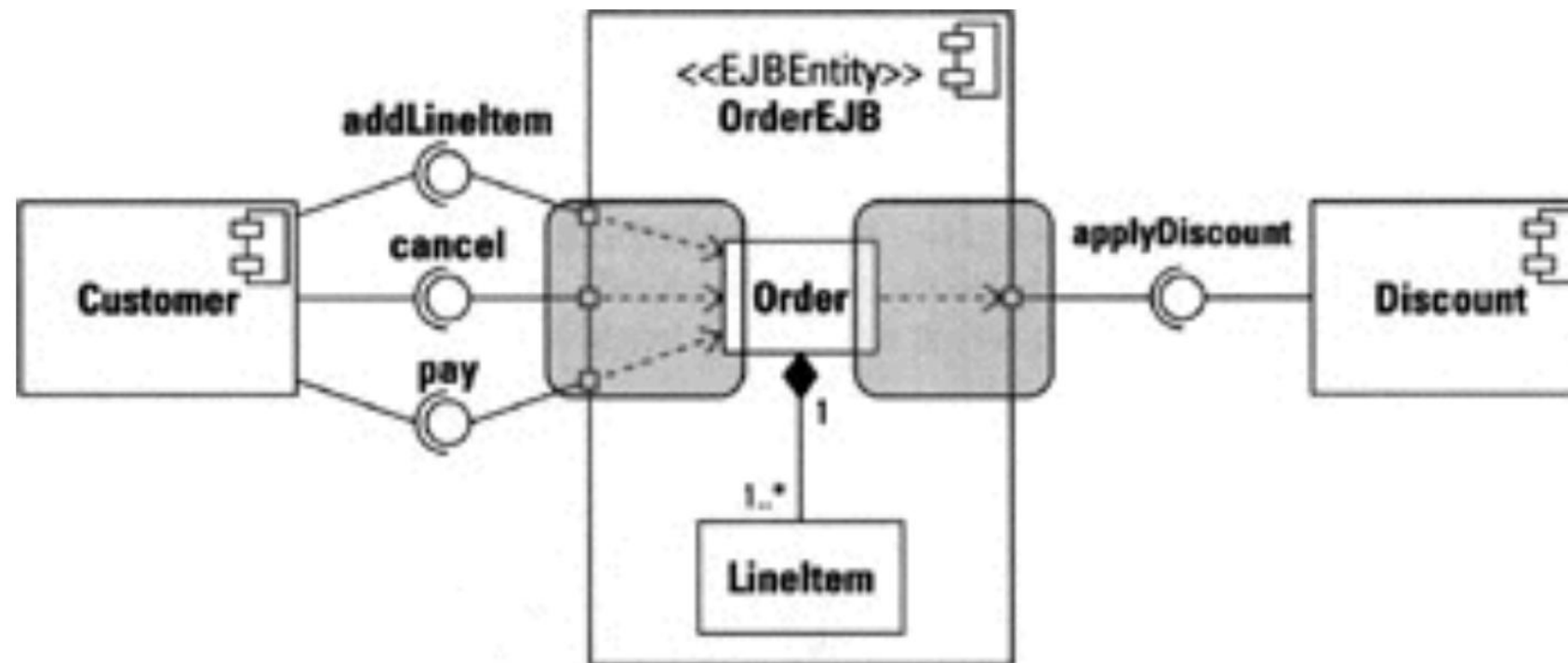


Sơ đồ thành phần

Các cổng ánh xạ các giao diện của thành phần tới các hiện thực hỗ trợ chúng

Một cổng xuất

hiện dưới dạng một hình vuông nhỏ ở cạnh của thành phần

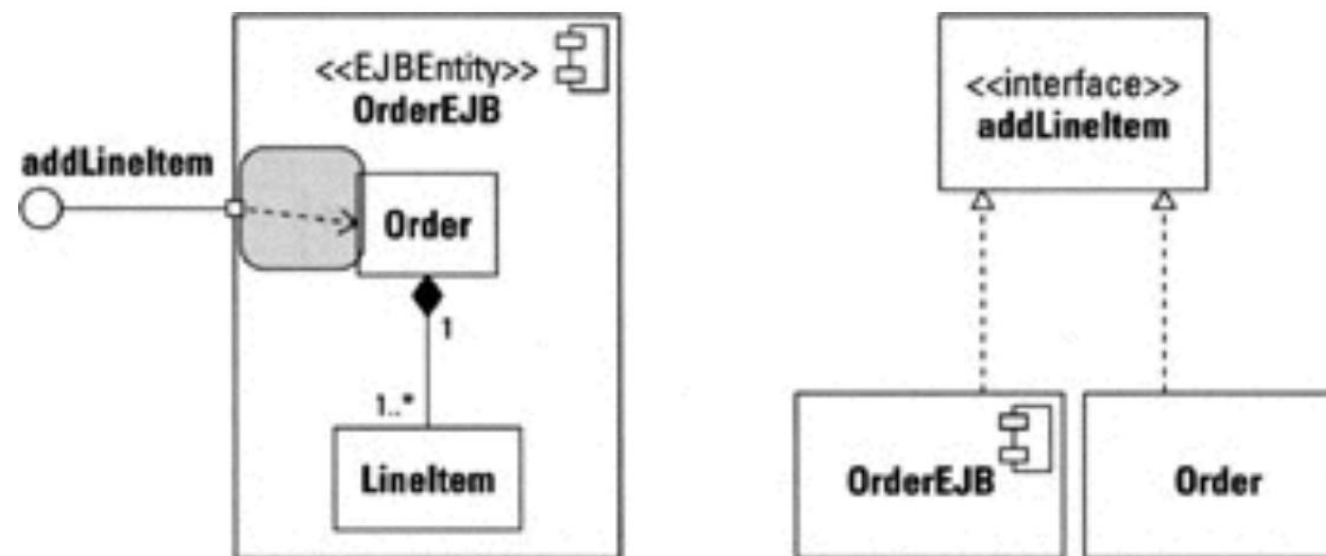


Sơ đồ thành phần

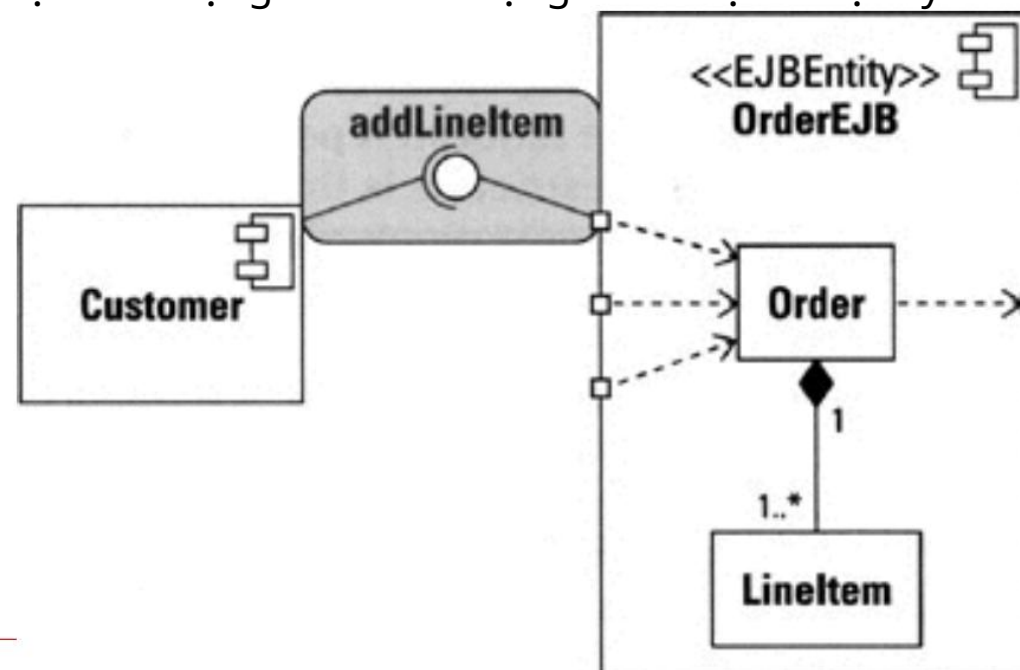
Trình kết nối là một liên kết cho phép giao tiếp giữa hai hoặc nhiều các thành phần

- Hai loại kết nối: đầu nối ủy quyền và đầu nối lắp ráp

Trình kết nối ủy quyền ánh xạ yêu cầu từ cổng này sang cổng khác hoặc tới một thực hiện cung cấp việc triển khai yêu cầu

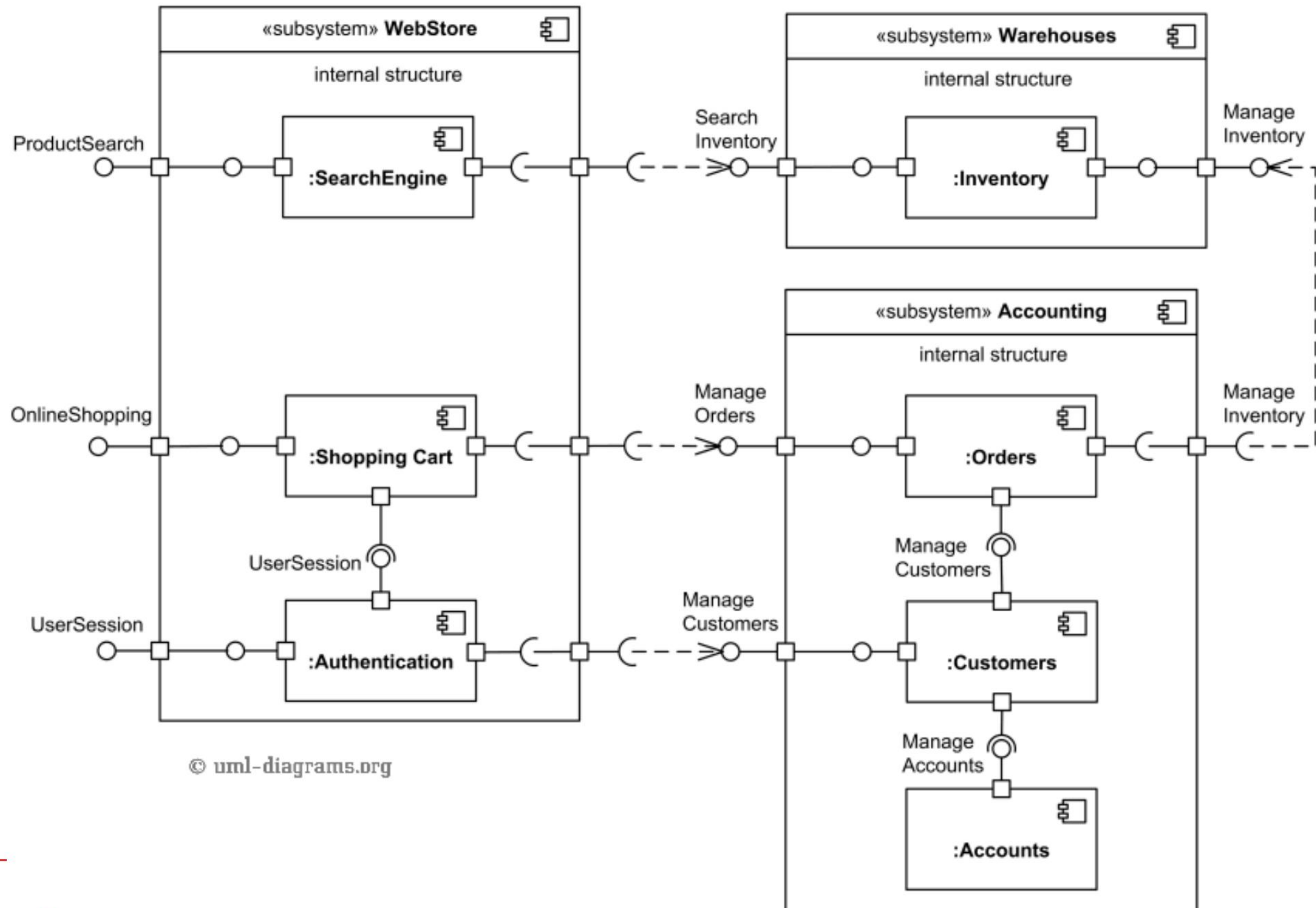


Một đầu nối lắp ráp được sử dụng để ánh xạ giao diện được yêu cầu tới một giao diện được cung cấp giao diện



Sơ đồ thành phần

Ví dụ: Ví dụ về mua sắm trực tuyến với ba hệ thống con liên quan -
Cửa hàng trực tuyến, kho bãi và kế toán



Sơ đồ triển khai

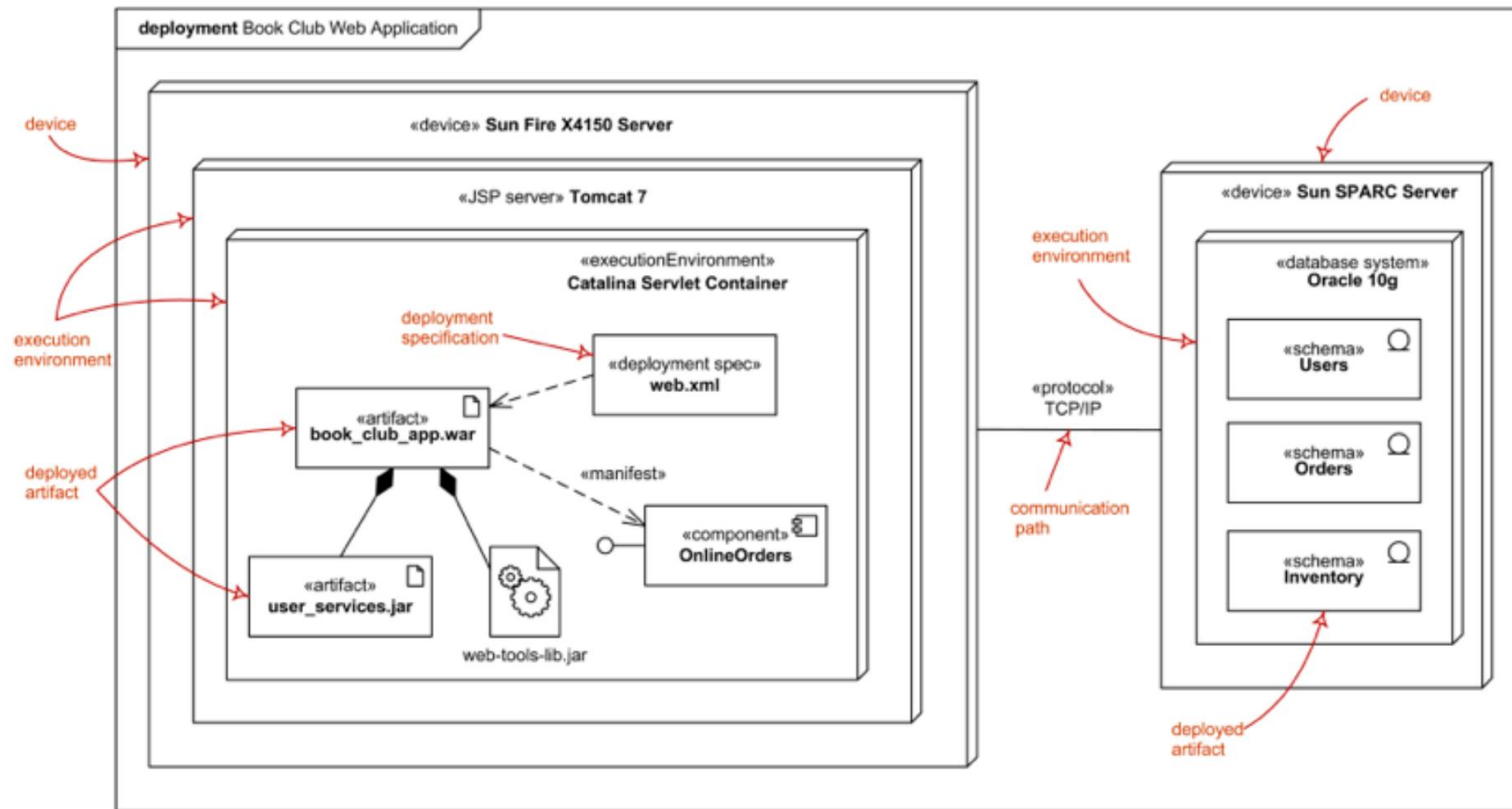
Sơ đồ triển khai thể hiện kiến trúc của một hệ thống như triển khai (phân phối) các tạo phẩm phần mềm cho các mục tiêu triển khai ■ Các tạo phẩm thể hiện các yếu tố cụ thể trong thế giới vật lý như tập tin thực thi, thư viện, kho lưu trữ, lược đồ cơ sở dữ liệu, tập tin cấu hình, .

- Mục tiêu triển khai thường được biểu diễn bằng một nút hoặc thiết bị phần cứng hoặc môi trường thực thi phần mềm. Các nút có thể được kết nối

Sơ đồ triển khai có thể mô tả kiến trúc ở mức đặc tả (cũng gọi là cấp độ loại) hoặc ở cấp độ phiên bản (tương tự như sơ đồ lớp và sơ đồ đối tượng).

Sơ đồ triển khai

Sơ đồ triển khai cấp độ đặc tả (còn gọi là cấp loại cuộc gọi) hiển thị một số tổng quan về việc triển khai các thành phần lạ cho các mục tiêu triển khai mà không tham chiếu đến các phiên bản cụ thể của thành phần hoặc nút.

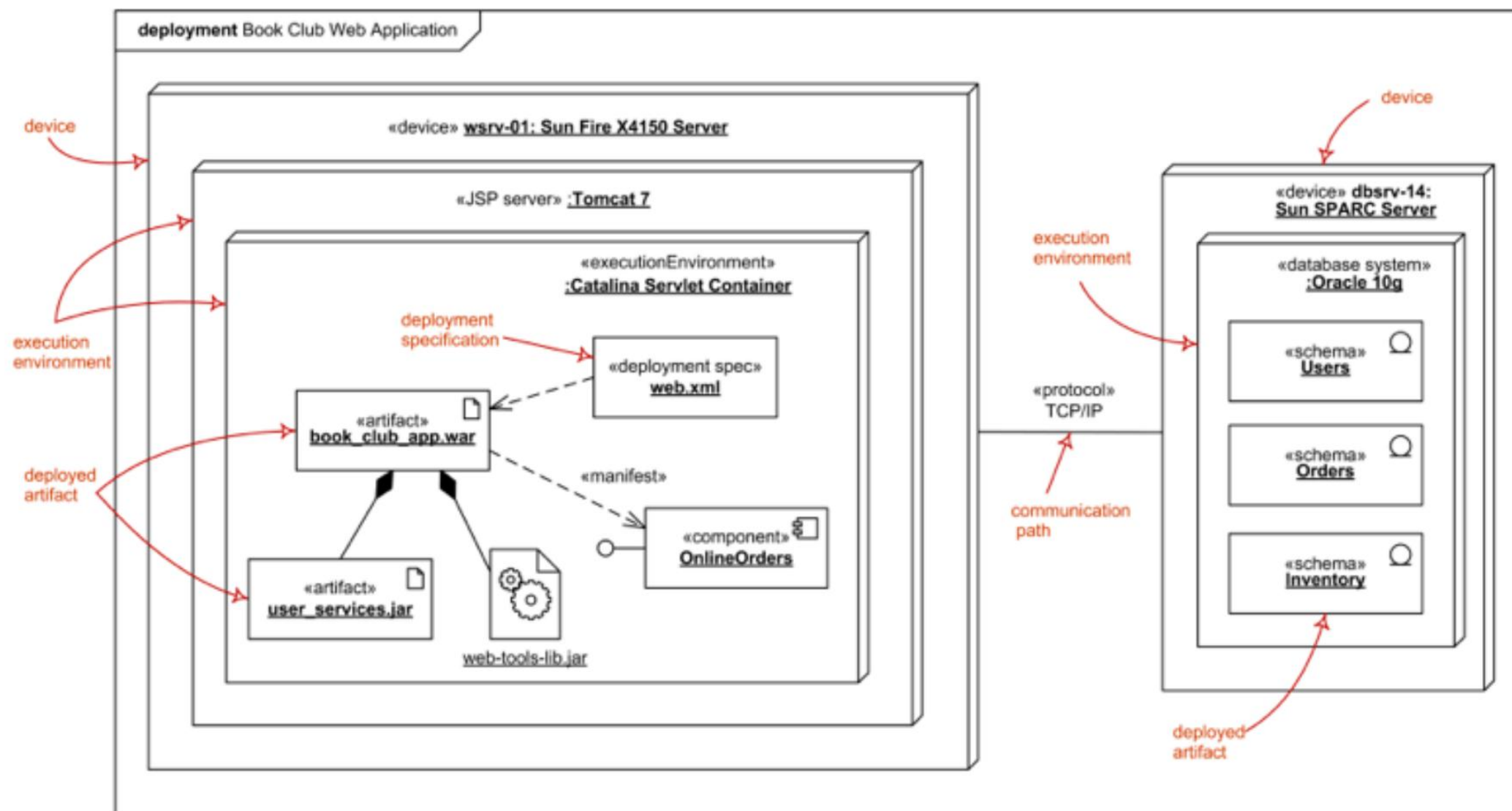


Sơ đồ triển khai mức đặc tả -

ứng dụng web được triển khai trên máy chủ Tomcat JSP và các lược đồ cơ sở dữ liệu - vào hệ thống cơ sở dữ liệu

Sơ đồ triển khai

Sơ đồ triển khai cấp độ phiên bản cho thấy việc triển khai các phiên bản của tạo tác cho các trường hợp cụ thể của mục tiêu triển khai. Ví dụ: có thể hữu ích khi hiển thị sự khác biệt trong việc triển khai cho các môi trường phát triển, dàn dựng hoặc sản xuất với tên/id cụ thể cho các dịch vụ hoặc thiết bị triển khai



Sơ đồ triển khai cấp độ phiên bản - ứng dụng web được triển khai cho máy chủ Tomcat JSP và các schema cơ sở dữ liệu - tới hệ thống cơ sở dữ liệu